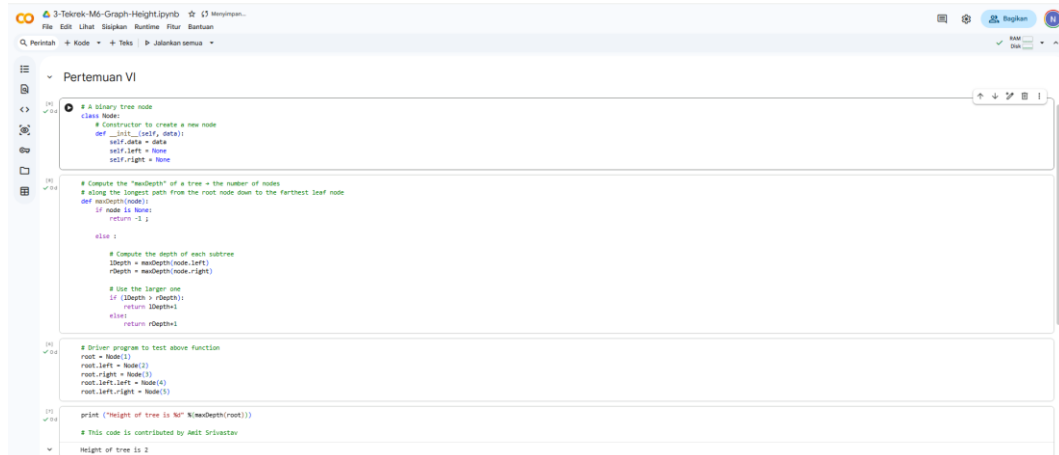


Nafil Fadillah
20123958
3KB02

Pertemuan Minggu Ke 6



```
# A binary tree node
class Node:
    # Constructor to create a new node
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Compute the "height" of a tree - the number of nodes
# along the longest path from the root node down to the farthest leaf node
def maxDepth(node):
    if node is None:
        return -1

    else:
        # Compute the depth of each subtree
        lDepth = maxDepth(node.left)
        rDepth = maxDepth(node.right)

        # Use the larger one
        if (lDepth > rDepth):
            return lDepth+1
        else:
            return rDepth+1

# Driver program to test above function
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)

print("Height of tree is %d" % maxDepth(root))

# This code is contributed by Aarti Srivastava
Height of tree is 2
```

Penjelasan Kode:

Program ini dibuat untuk menghitung tinggi (height) dari sebuah binary tree. Binary tree sendiri adalah struktur data berbentuk pohon, di mana setiap node memiliki maksimal dua anak, yaitu kiri dan kanan.

Di awal program, dibuat sebuah “cetakan” node menggunakan class Node. Setiap node menyimpan sebuah nilai (data) dan dua referensi, yaitu ke anak kiri (left) dan anak kanan (right). Saat node pertama kali dibuat, kedua anak tersebut masih kosong (None), artinya node tersebut belum terhubung ke node lain.

Selanjutnya terdapat fungsi maxDepth yang berperan sebagai inti dari program. Fungsi ini digunakan untuk mencari kedalaman maksimum (tinggi) dari tree. Cara kerjanya menggunakan konsep rekursi, yaitu fungsi yang memanggil dirinya sendiri.

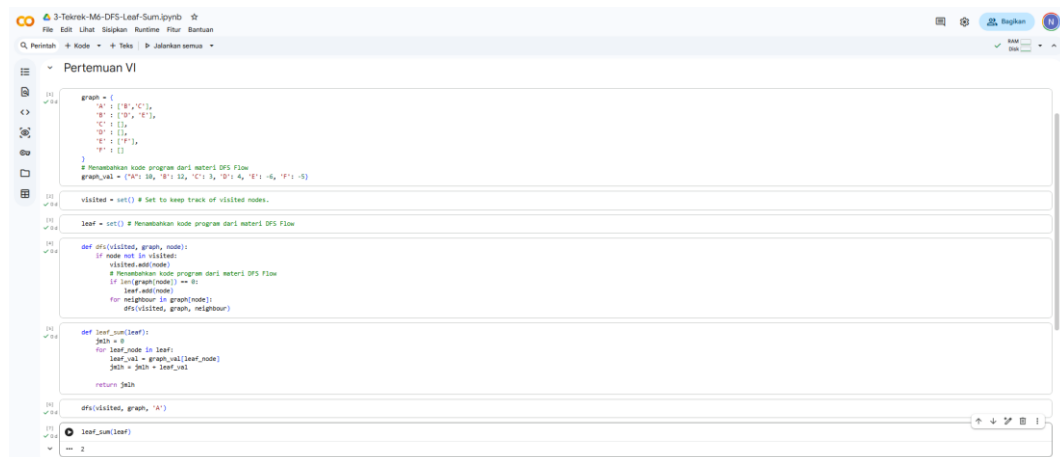
Ketika fungsi ini dijalankan, pertama-tama ia akan mengecek apakah node yang sedang diproses itu kosong. Jika kosong, maka fungsi mengembalikan nilai -1. Nilai ini digunakan sebagai dasar perhitungan tinggi ketika tidak ada node sama sekali.

Jika node tidak kosong, maka fungsi akan menghitung kedalaman dari subtree kiri dan subtree kanan dengan cara memanggil dirinya sendiri pada masing-masing anak node. Proses ini akan terus berlanjut ke bawah sampai mencapai node paling ujung (leaf).

Setelah mendapatkan kedalaman kiri dan kanan, fungsi akan membandingkan keduanya. Nilai yang lebih besar dipilih, kemudian ditambahkan 1 untuk menghitung level node saat ini. Dengan cara ini, program secara bertahap menghitung jalur terpanjang dari root hingga ke leaf.

Pada bagian berikutnya, program membuat contoh struktur binary tree secara manual. Node dengan nilai 1 dijadikan sebagai root, kemudian ditambahkan anak kiri (2) dan kanan (3). Node 2 masih memiliki dua anak lagi, yaitu 4 dan 5. Struktur ini membentuk sebuah pohon dengan beberapa level.

Terakhir, program memanggil fungsi maxDepth dengan parameter root, lalu menampilkan hasilnya. Hasil yang diperoleh adalah 2, yang menunjukkan bahwa jalur terpanjang dari root ke leaf memiliki panjang 2 edge (misalnya dari node 1 ke 2 ke 4, atau 1 ke 2 ke 5).



```
graph = {
    'A': ['B','C'],
    'B': ['D','E'],
    'C': [],
    'D': [],
    'E': ['F'],
    'F': []
}

# Memasukkan kode program dari materi DFS Flow
graph_val = ['A': 10, 'B': 10, 'C': 10, 'D': 0, 'E': 0, 'F': 0]

visited = set() # Set to keep track of visited nodes.

leaf = set() # Memasukkan kode program dari materi DFS Flow

def dfs(visited, graph, node):
    if node not in visited:
        visited.add(node)
        # Memasukkan kode program dari materi DFS Flow
        if len(graph[node]) == 0:
            leaf.add(node)
            for neighbour in graph[node]:
                dfs(visited, graph, neighbour)

def leaf_sum(leaf):
    sum = 0
    for leaf_node in leaf:
        leaf_val = graph_val[leaf_node]
        sum = sum + leaf_val
    return sum

dfs(visited, graph, 'A')

leaf_sum(leaf)

2
```

Penjelasan Kode:

Program ini dibuat untuk menelusuri sebuah graph menggunakan metode Depth First Search (DFS), kemudian menentukan node mana saja yang termasuk leaf (node tanpa anak), dan akhirnya menjumlahkan nilai dari node-node leaf tersebut.

Di awal program, didefinisikan sebuah graph menggunakan struktur dictionary. Graph ini merepresentasikan hubungan antar node, di mana setiap node memiliki daftar tetangga (anak). Misalnya node A terhubung ke B dan C, lalu B terhubung ke D dan E, dan seterusnya. Ada beberapa node seperti C, D, dan F yang tidak memiliki anak, sehingga mereka berpotensi menjadi leaf node.

Selain itu, program juga mendefinisikan nilai untuk setiap node dalam dictionary terpisah. Nilai-nilai ini nantinya akan digunakan untuk proses penjumlahan, khususnya untuk node-node yang termasuk leaf.

Selanjutnya, disiapkan dua variabel penting, yaitu visited dan leaf. Variabel visited digunakan untuk mencatat node yang sudah dikunjungi selama proses DFS

agar tidak terjadi pengulangan. Sementara itu, leaf digunakan untuk menyimpan node-node yang teridentifikasi sebagai leaf.

Fungsi utama dalam program ini adalah fungsi DFS. Fungsi ini bekerja secara rekursif, yaitu memanggil dirinya sendiri untuk menelusuri graph secara mendalam. Prosesnya dimulai dari suatu node (dalam hal ini node A), kemudian node tersebut ditandai sebagai sudah dikunjungi. Setelah itu, program mengecek apakah node tersebut memiliki anak atau tidak. Jika tidak memiliki anak (daftar tetangganya kosong), maka node tersebut dikategorikan sebagai leaf dan dimasukkan ke dalam himpunan leaf.

Jika node masih memiliki anak, maka fungsi akan melanjutkan penelusuran ke setiap anaknya. Proses ini berlangsung terus hingga semua node yang terhubung telah dikunjungi. Dengan cara ini, seluruh graph dapat ditelusuri secara menyeluruh, dan semua leaf node berhasil dikumpulkan.

Setelah proses DFS selesai, program beralih ke fungsi berikutnya yaitu `leaf_sum`. Fungsi ini bertugas untuk menghitung total nilai dari semua node yang termasuk leaf. Caranya adalah dengan melakukan iterasi pada setiap node dalam himpunan leaf, lalu mengambil nilai node tersebut dari dictionary `graph_val`, dan menjumlahkannya satu per satu hingga diperoleh total akhir.

Pada graph yang digunakan, node yang termasuk leaf adalah C, D, dan F karena ketiganya tidak memiliki anak. Nilai dari masing-masing node tersebut adalah 3, 4, dan -5. Ketika dijumlahkan, hasilnya adalah 2.

Di bagian akhir, program menjalankan fungsi DFS dengan titik awal node A untuk menelusuri seluruh graph, kemudian memanggil fungsi `leaf_sum` untuk menghitung total nilai leaf. Hasil yang ditampilkan adalah angka 2, yang merupakan jumlah dari semua nilai node leaf.